TRAVAUX PRATIQUES MLOps

Chapitre 3 — Versioning et suivi des modèles avec MLflow

Objectif : entraîner plusieurs modèles, suivre leurs paramètres et métriques, comparer les expériences et recharger un modèle enregistré.

Durée indicative : 2 h à 3 h
Environnement : Windows + PowerShell, Python 3.10+ recommandé

Support pédagogique — pas à pas

1. Objectifs pédagogiques

À la fin du TP, l'étudiant saura :

• Créer un environnement Python isolé et installer MLflow.

• Entraîner deux modèles de classification avec scikit-learn.

• Enregistrer les paramètres, métriques et artefacts dans MLflow Tracking.

• Comparer les expériences dans l'interface Web de MLflow.

• Recharger un modèle à partir de son identifiant de run.

2. Préparer l'environnement

Étape 1 — Créer le dossier et l'environnement virtuel

```
mkdir TP_MLflow
cd TP_MLflow
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Si PowerShell bloque l'activation, exécutez cette commande dans la même fenêtre, puis réessayez :

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

Étape 2 — Installer les dépendances

```
python -m pip install --upgrade pip
pip install mlflow scikit-learn pandas
```

Vérifiez l'installation :

```
python -c "import mlflow, sklearn; print('MLflow', mlflow.__version__); print('scikit-learn', sklearn.__version__)
```

3. Créer le script d'entraînement

Créez un fichier nommé **train.py** dans le dossier TP_MLflow. Le script utilise le jeu de données Iris et entraîne deux forêts aléatoires avec des valeurs différentes du paramètre n_estimators.

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

mlflow.set_experiment("TP_MLflow_Iris")

for n in [50, 100]:
    with mlflow.start_run(run_name=f"RF_{n}"):
        model = RandomForestClassifier(n_estimators=n, random_state=42)
        model.fit(X_train, y_train)
        predictions = model.predict(X_test)

        mlflow.log_param("n_estimators", n)
        mlflow.log_param("test_size", 0.2)
        mlflow.log_metric("accuracy", accuracy_score(y_test, predictions))
        mlflow.log_metric(
            "f1_macro", f1_score(y_test, predictions, average="macro")
        )
        mlflow.sklearn.log_model(sk_model=model, name="iris_model")
        print(f"Run RF_{n} terminé")
```

4. Exécuter l'entraînement

```
python train.py
```

Résultat attendu : deux runs sont créés, RF_50 et RF_100. Chaque run contient les paramètres, les métriques et le modèle enregistré comme artefact.

5. Explorer MLflow Tracking

Étape 1 — Démarrer l'interface

```
mlflow ui
```

Ouvrez ensuite **http://127.0.0.1:5000** dans votre navigateur. Gardez le terminal ouvert pendant l'utilisation de l'interface.

Étape 2 — Comparer les runs

• Ouvrez l'expérience « TP_MLflow_Iris ».

• Sélectionnez les runs RF_50 et RF_100.

• Comparez les paramètres n_estimators et test_size.

• Comparez les métriques accuracy et f1_macro.

• Ouvrez un run pour examiner ses artefacts et le modèle sauvegardé.

Les résultats exacts peuvent varier selon l'environnement et les versions des bibliothèques. L'objectif est de comprendre la traçabilité, pas d'obtenir une valeur précise.

6. Recharger un modèle enregistré

Ajoutez un fichier **load_model.py**. Remplacez RUN_ID par l'identifiant d'un run affiché dans MLflow.

```
import mlflow.sklearn
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

RUN_ID = "REMPLACER_PAR_LE_RUN_ID"
model_uri = f"runs:/{RUN_ID}/iris_model"

model = mlflow.sklearn.load_model(model_uri)
X, y = load_iris(return_X_y=True)
predictions = model.predict(X)
print("Accuracy sur Iris complet :", accuracy_score(y, predictions))
```

Exécutez :

```
python load_model.py
```

Remarque : cette vérification utilise ici l'ensemble Iris complet pour illustrer le chargement. Pour une évaluation scientifique, conservez le même jeu de test que lors de l'entraînement.

7. Défis complémentaires

• Ajouter n_estimators = 150 et comparer les trois runs.

• Tester max_depth = 3 puis max_depth = None.

• Enregistrer la durée d'entraînement comme métrique.

• Créer une expérience distincte pour une régression ou un autre jeu de données.

• Expliquer pourquoi conserver les paramètres, métriques et artefacts facilite la reproductibilité.

8. Questions de validation

1. Quelle différence existe entre une expérience et un run dans MLflow ?

2. Quels paramètres et quelles métriques le script enregistre-t-il ?

3. Où peut-on retrouver le modèle sauvegardé ?

4. Pourquoi faut-il conserver l'identifiant du run ?

5. Quelle est la différence entre MLflow Tracking et un registre de modèles ?

9. Barème indicatif

Critère	Points
Environnement configuré et dépendances installées	2
Script exécuté sans erreur	3
Paramètres et métriques enregistrés	3
Comparaison des runs dans l'interface	1
Chargement du modèle et réponses aux questions	1
Total	**10**

Livrables à remettre : fichiers train.py et load_model.py, captures d'écran des deux runs, tableau comparatif des métriques et réponses aux questions.